Transformers For Vision

Lecture 8

Transformer

- Non-vision specific model
  - Typically applied to 1-D sequence data
- Transformer “encoder”
  - A stack of alternating self-attention and MLP blocks
  - Residuals and LayerNorm
- Transformer “decoder” (not shown)
  - A slightly more involved architecture useful when the output space is different from the input space (e.g. translation)

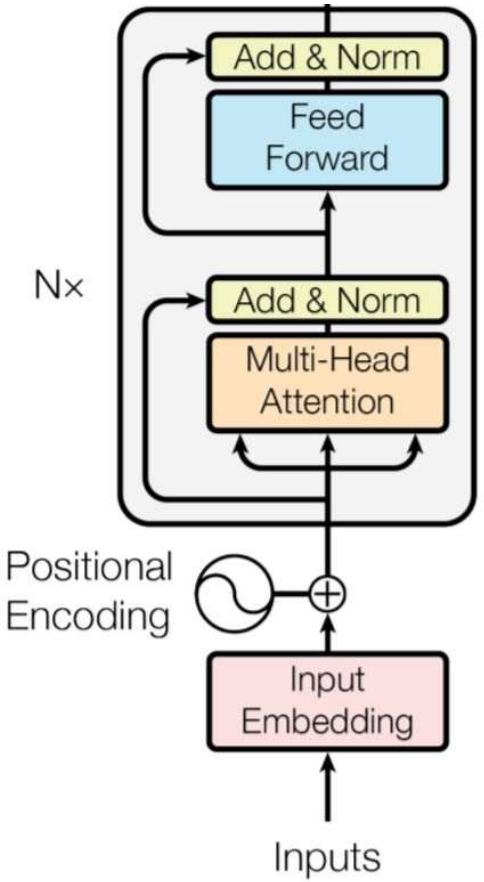
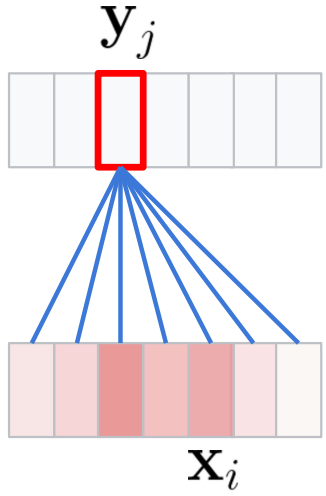


Figure from Vaswani et al.

Vaswani et al., *Attention Is All You Need*, NeurIPS 2017

Self-attention

- Each of the tokens (=vectors) attends to all tokens
  - Extra tricks: learned key, query, and value projections, inverse-sqrt scaling in the softmax, and multi-headed attention (omit for simplicity)
- It's a set operation (permutation-invariant)
  - ...and hence need "position embeddings" to "remember" the spatial structure
- It's a global operation
  - Aggregates information from all tokens



$$\boldsymbol{\alpha}_j = softmax(\frac{K\mathbf{x}_1 \cdot Q\mathbf{x}_j}{\sqrt{d_K}}, \ldots, \frac{K\mathbf{x}_n \cdot Q\mathbf{x}_j}{\sqrt{d_K}})$$

$$\mathbf{y}_j = \sum_{i=1}^{n} \alpha_{ji} V\mathbf{x}_i$$

Simplified! Multi-headed attention not shown

Vaswani et al., *Attention Is All You Need*, NeurIPS 2017

Self-Attention with Queries, Keys, Values

Make three version of each input embedding x(i)

- **Query** vector $\mathbf{q}^{(i)} = \mathbf{W}_q \mathbf{x}^{(i)}$
- **Key** vector: $\mathbf{k}^{(i)} = \mathbf{W}_k \mathbf{x}^{(i)}$
- **Value** vector: $\mathbf{v}^{(i)} = \mathbf{W}_v \mathbf{x}^{(i)}$

The **attention weight of the *j*-th position** to compute the **new output for the *i*-th position** depends on the **query of i** and the **key of j (scaled):**

$$w_j^{(i)} = \frac{\exp(\mathbf{q}^{(i)}\mathbf{k}^{(j)})/\sqrt{k}}{\sum_j (\exp(\mathbf{q}^{(i)}\mathbf{k}^{(j)})/\sqrt{k})}$$

The **new output vector for the i-th position** depends on the **attention weights** and **value** vectors of all **input positions j**:

$$\mathbf{y}^{(i)} = \sum_{j=1..T} w_j^{(i)} \mathbf{v}^{(j)}$$

[Julia Hockenmaier, Lecture 9, UIUC]

Transformer self-attention layer

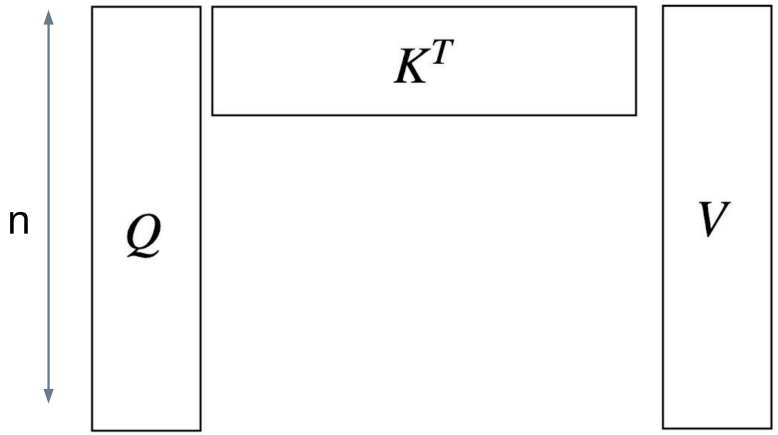
Input: X (matrix of n embedding vectors, each dim m)

Parameters (learned): W_Q, W_K, W_V

Compute:

$$Q = XW_Q$$
$$K = XW_K$$
$$V = XW_V$$

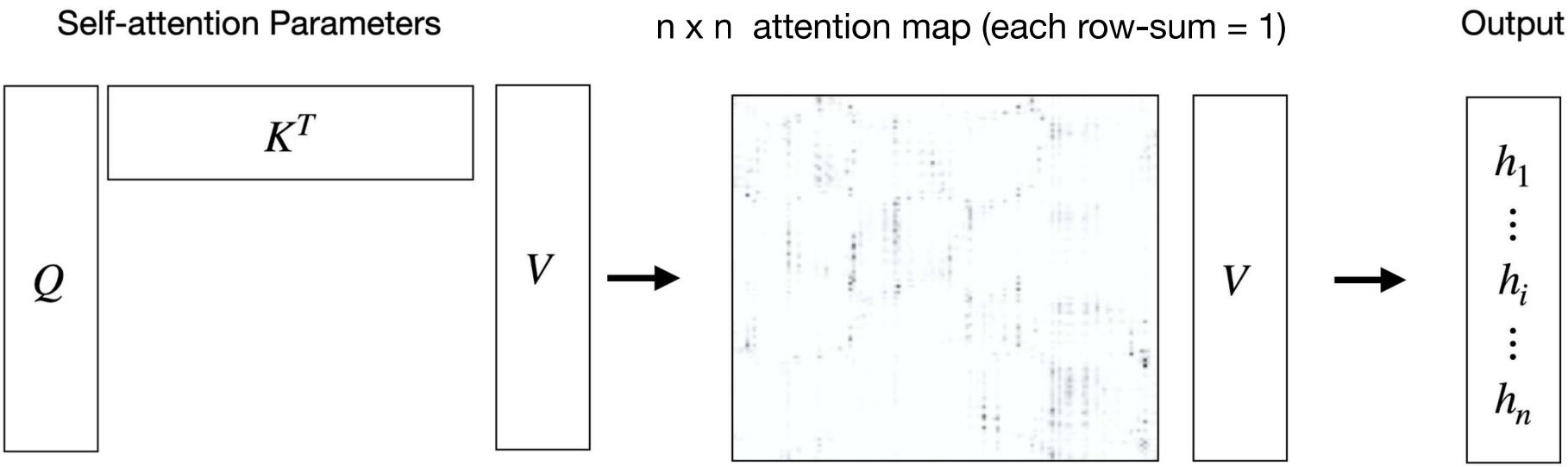
Self-attention Parameters



$$Q, K, V \in \mathbb{R}^{n \times m}$$

$$QK^T \in \mathbb{R}^{n \times n}$$

Transformer self-attention

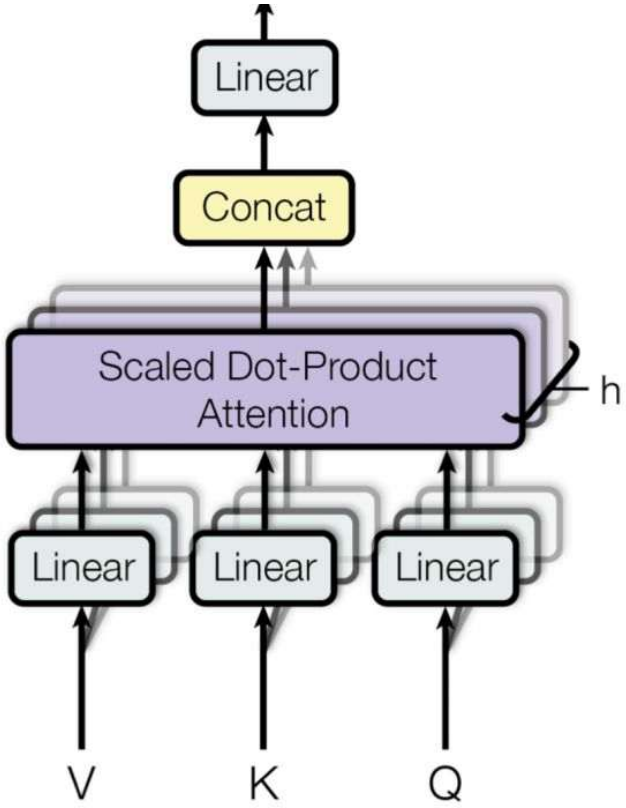


Output matrix H = $\mathrm{softmax}(\frac{1}{\sqrt{d}} QK^T) \cdot V$

Self-attention explicitly models interactions between all pairs of input embeddings

Multi-Head attention

- Learn *h* different linear projections of Q,K,V
- Compute attention separately on each of these *h* versions
- Concatenate and project the resultant vectors to a lower dimensionality.
- Each attention head can use low dimensionality



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$

$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

[Julia Hockenmaier, Lecture 9, UIUC]

Positional Encoding (1-D)

How to capture sequence order?

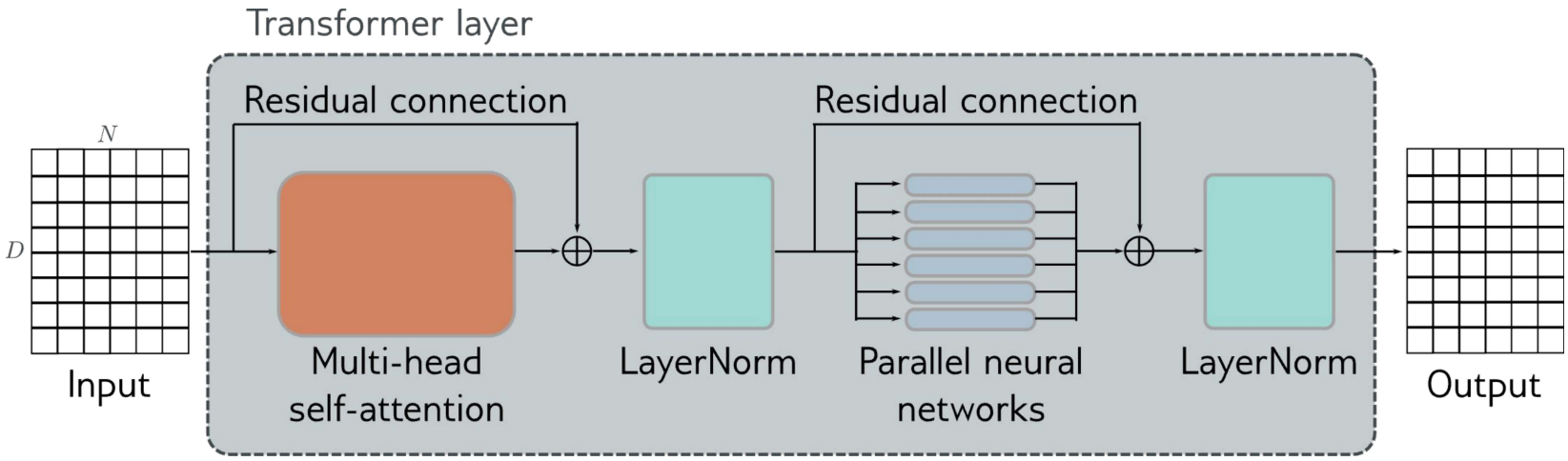
Add positional embeddings to input embeddings

- Same dimension
- Can be learned or fixed

Fixed encoding: sin / cos of different frequencies:

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

Transformer Layer -- Complete



Transform Layer

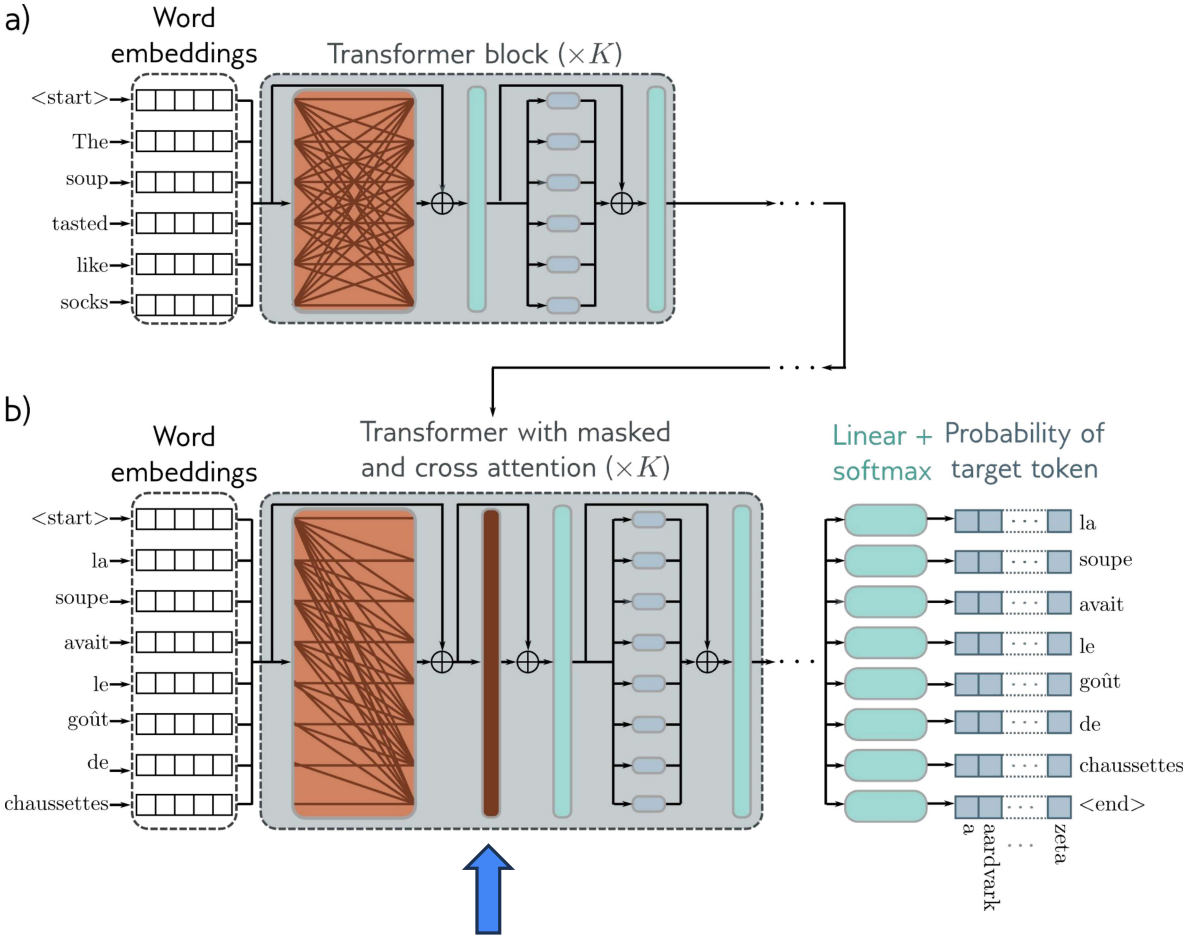
$$
\begin{aligned}
\mathbf{X} &\leftarrow \mathbf{X} + \mathbf{MhSa}[\mathbf{X}] \\
\mathbf{X} &\leftarrow \mathbf{LayerNorm}[\mathbf{X}] \\
\mathbf{x}_n &\leftarrow \mathbf{x}_n + \mathbf{mlp}[\mathbf{x}_n] \\
\mathbf{X} &\leftarrow \mathbf{LayerNorm}[\mathbf{X}],
\end{aligned}
$$

LayerNorm

$$y = \frac{x - \mathbf{E}[x]}{\sqrt{\mathbf{Var}[x] + \epsilon}} * \gamma + \beta$$

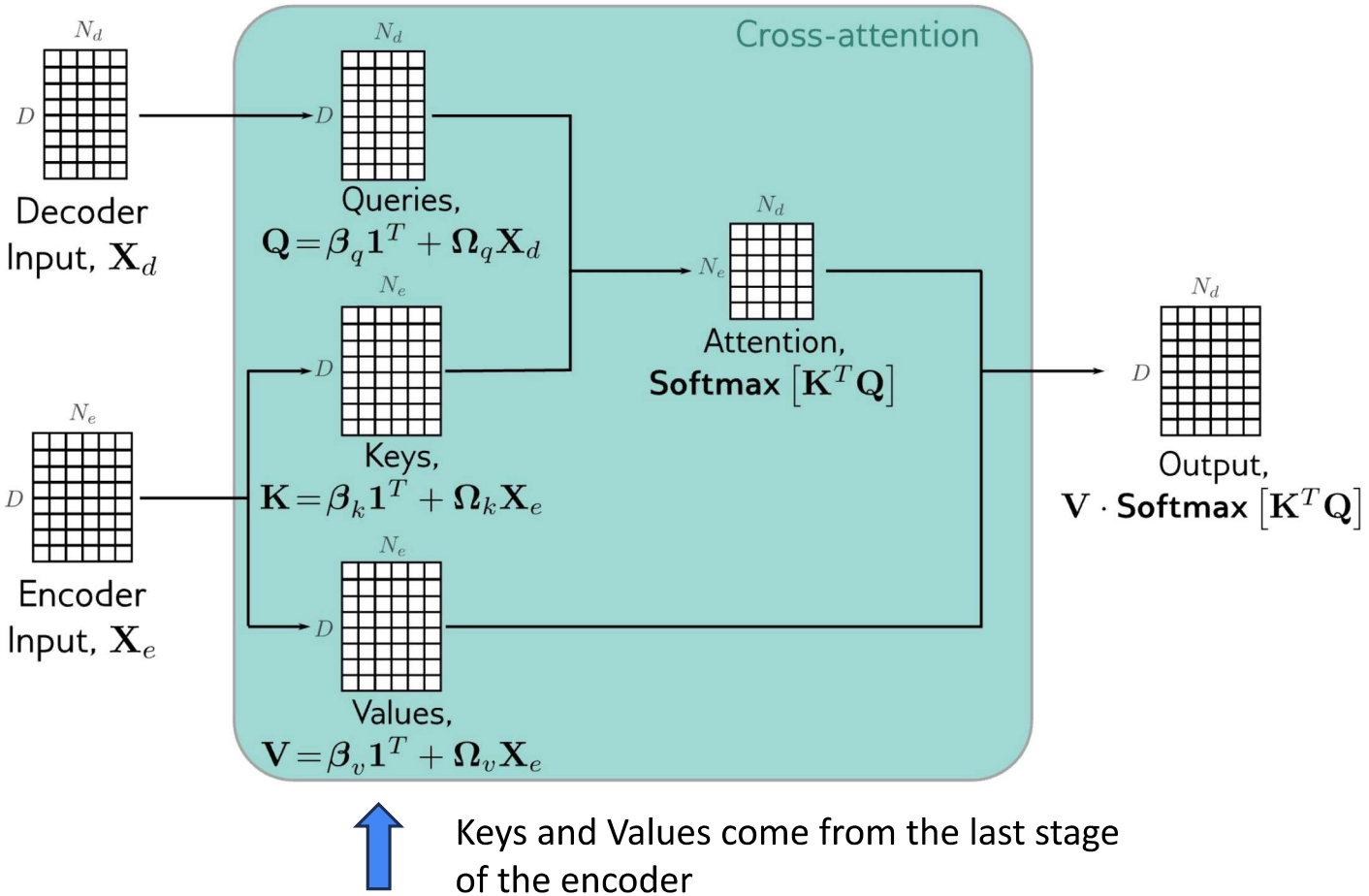
Calculated column-wise

Encoder Decoder Model



- The transformer layer in the decoder of the encoder-decoder model has an extra stage
- Attends to the input of the encoder with *cross attention* using Keys and Values from the output of the encoder

Cross-Attention



Keys and Values come from the last stage of the encoder

BERT model in NLP

- Transformers pre-trained self-supervised perform great on many NLP tasks
  - Masked language modeling (MLM)
  - Next sentence prediction (NSP)

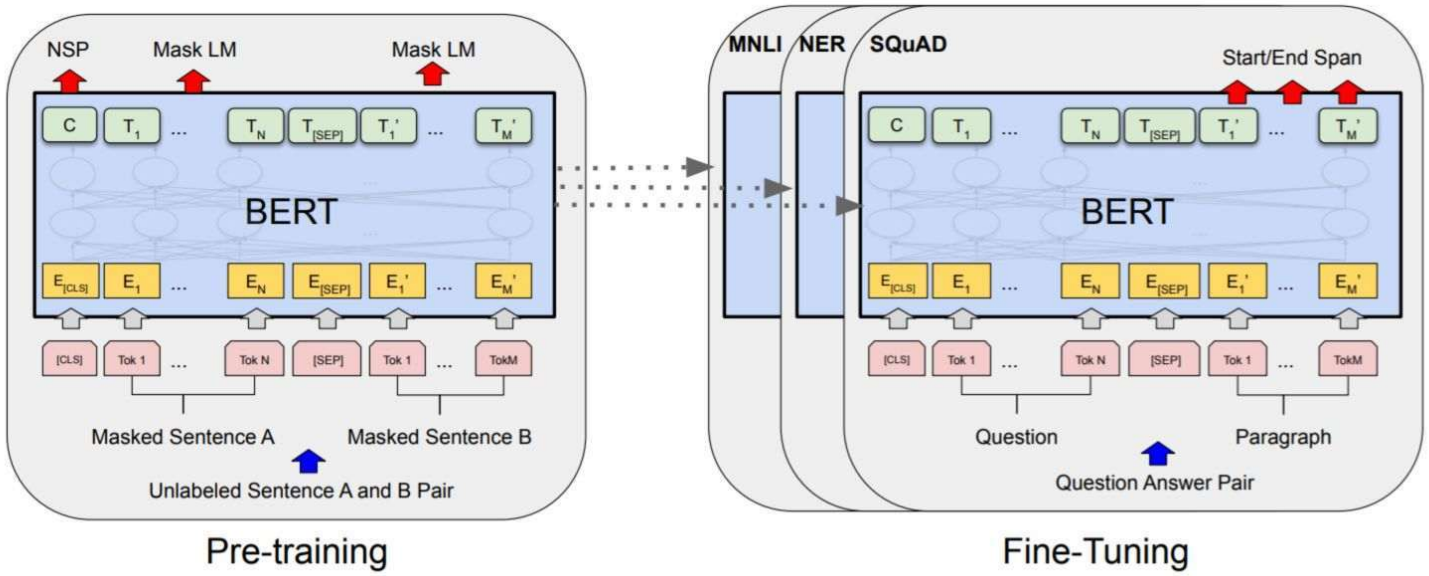
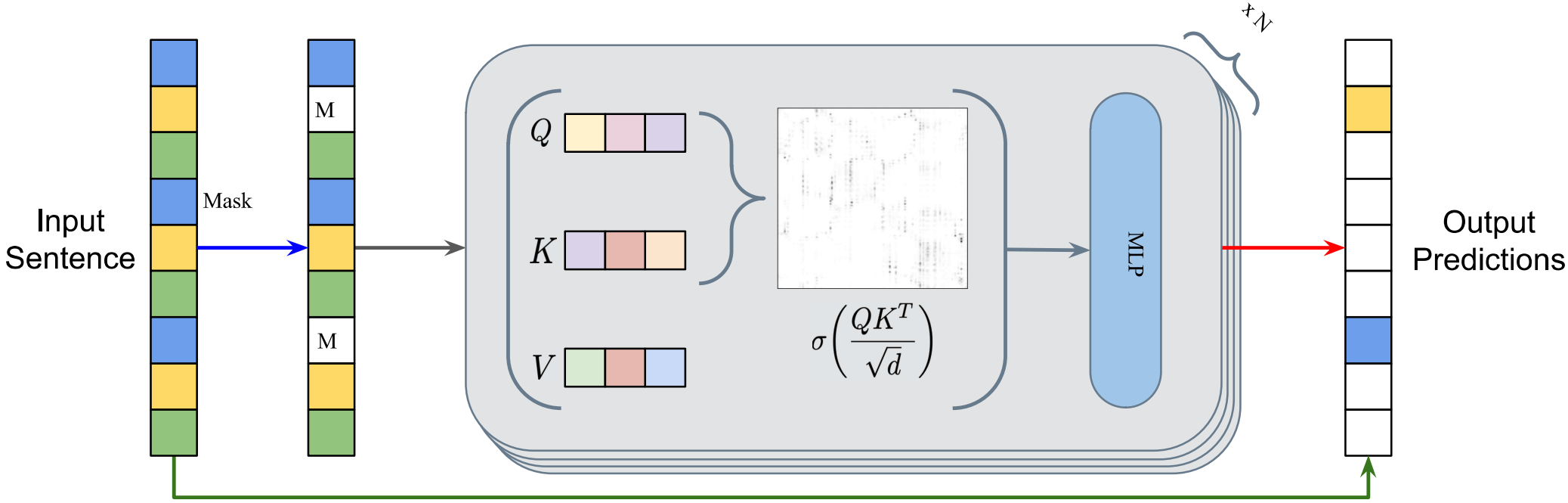


Figure from Devlin et al.

Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, arXiv 2018

Masked language modeling with Transformers (in NLP)



Training: Predict Masked Tokens

$$\mathcal{L}_{\text{MLM}}(X;\theta) = \mathop{\mathbb{E}}_{x\sim X} \mathop{\mathbb{E}}_{\text{mask}} \sum_{i\in\text{mask}} \log p(x_i | x_{j\notin\text{mask}};\theta)$$

(mask 15% at a time)

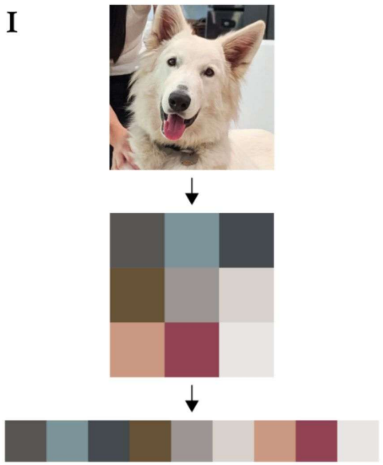
Transformers for vision?

- “LSTM → Transformer” ~ “ConvNet → ??? ”
- Issue with self-attention for vision: computation is quadratic in the input sequence length, quickly gets very expensive (with > few thousand tokens)
  - For ImageNet: 224x224 pixels → ~50,000 sequence length
  - Even worse for higher resolution and video

How can we deal with this quadratic complexity?

Image GPT – Inputs

- Reduced resolution to reduce context size: 32×32, 48×48 or 64×64
- Also reduced color palette from $3\times8 = 24$ bit to a 9-bit (512 colors) color palette by clustering (R, G, B) pixels with $k = 512$.

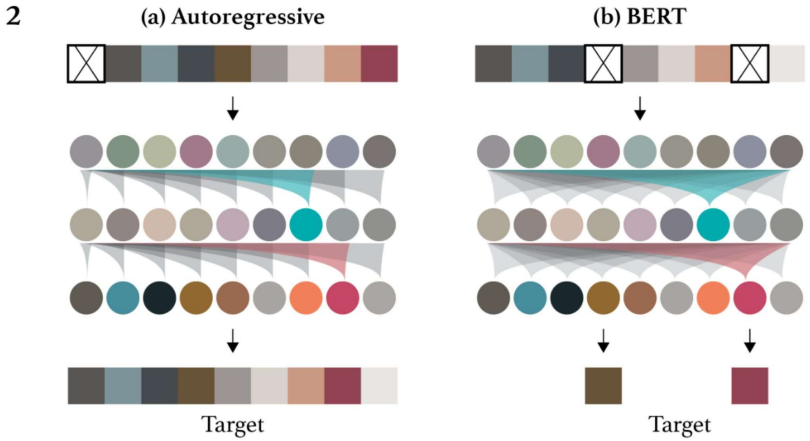


- Reduced resolution to reduce context size: 32×32, 48×48 or 64×64
- Also reduced color palette from $3\times8 = 24$ bit to a 9-bit (512 colors) color palette by clustering (R, G, B) pixels with $k = 512$.

https://openai.com/research/image-gpt
https://github.com/openai/image-gpt (deprecated)
https://huggingface.co/docs/transformers/model_doc/imagegpt

M. Chen *et al.*, "Generative Pretraining from Pixels," OpenAI, Technical Report, Jun. 2020.

Image GPT – Training Objectives



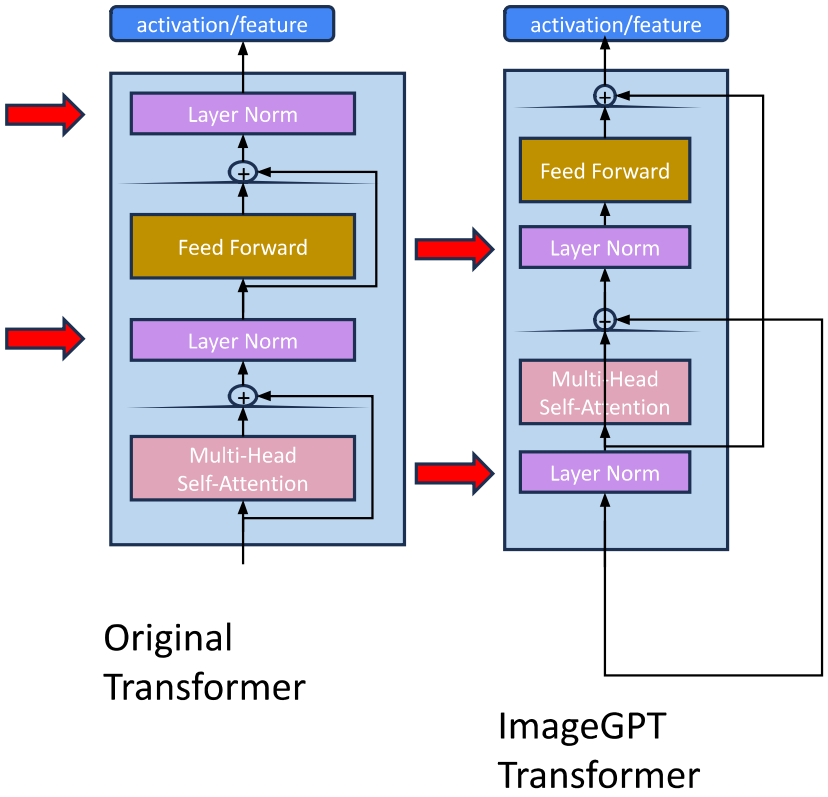
- Tried training with either *Autoregressive* or *BERT* style training objective

https://openai.com/research/image-gpt
https://github.com/openai/image-gpt (deprecated)
https://huggingface.co/docs/transformers/model_doc/imagegpt

M. Chen *et al.*, "Generative Pretraining from Pixels," OpenAI, Technical Report, Jun. 2020.

Image GPT – Transformer Layer

OpenAI

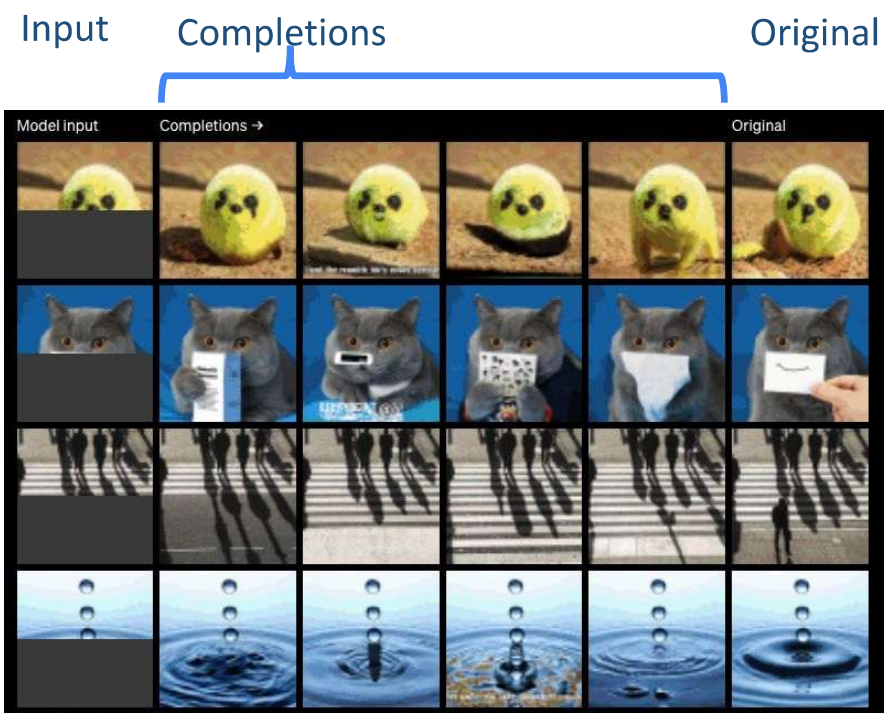


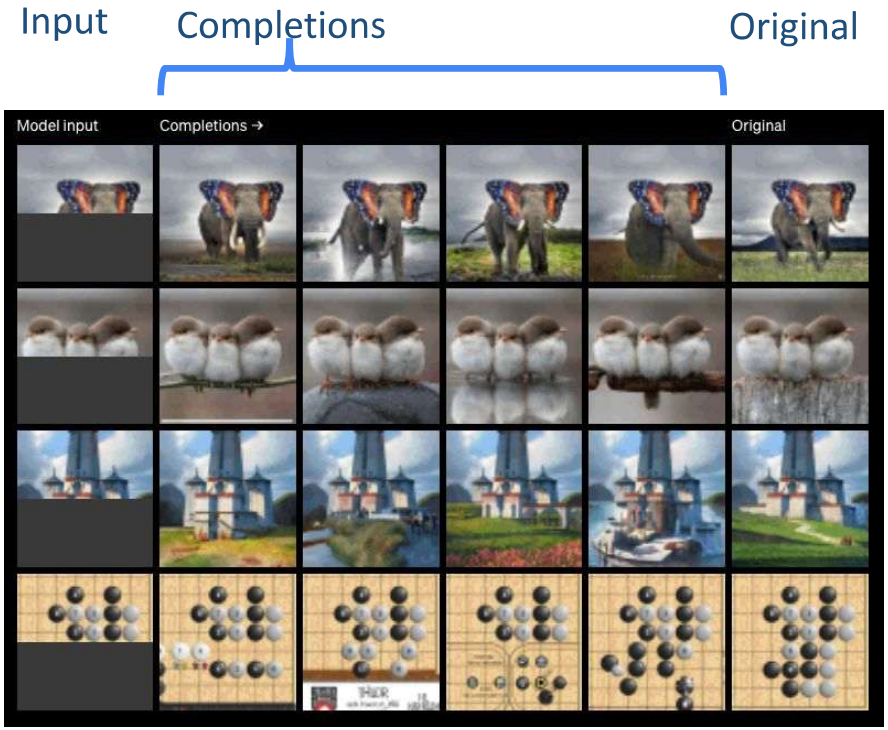
Original Transformer

ImageGPT Transformer

- LayerNorm moved to precede Self-Attention and Feed Forward block
- In the residual path

Image GPT – AR Pixel Prediction Results





https://openai.com/research/image-gpt

Image GPT – Pros and Cons

Pro:

- Gave insights into the representational power of Transformers with unsupervised training

Con:

- Worked on downscaled images of size 32x32 to 64x64

Vision Transformer (ViT) – June 2021

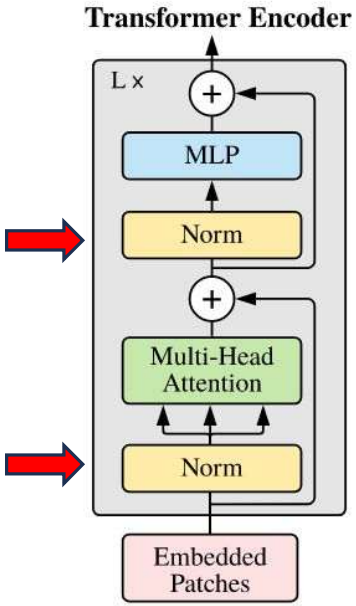
- Overcomes resolution limitation of ImageGPT by using patches
- Follows scalable NLP Transformer architectures to benefit from efficient implementations
- ImageNet Top-1 accuracy: 88.55%
- Performs poorly if just trained on ImageNet
  - can be expected since Transformers lack the inductive bias of CNNs
- Competitive when pre-trained on very large datasets (e.g. 14M – 300M) images – all supervised at this point

Large scale training trumps inductive bias???

https://arxiv.org/abs/2010.11929v2
https://github.com/google-research/vision_transformer

Be wary of method claims based on different data sets and compute.

Vision Transformer (ViT) – June 2021



- Uses same Transformer layer as ImageGPT and scalable NLP Transformers

https://arxiv.org/abs/2010.11929v2
https://github.com/google-research/vision_transformer

ViT: Putting it all together

1. Divide image into $P \times P$ patches



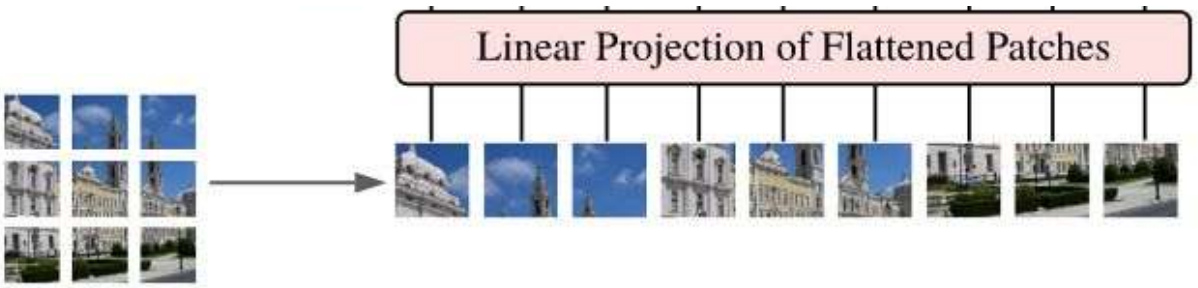
ViT: Putting it all together

1. Divide image into $P \times P$ patches
2. Create sequence of length $N = HW/P^2$



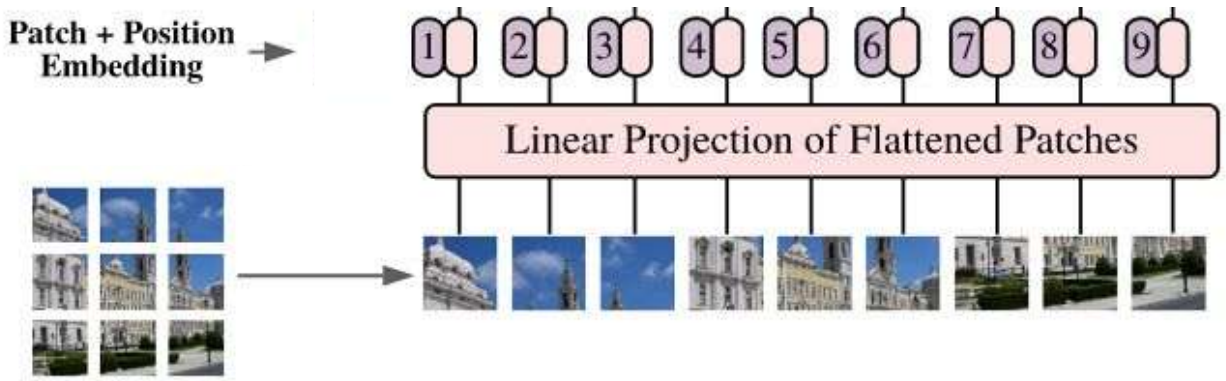
ViT: Putting it all together

1. Divide image into $P \times P$ patches
2. Create sequence of length $N = HW/P^2$
3. Flatten the patches and map to D dimensions with a trainable linear projection



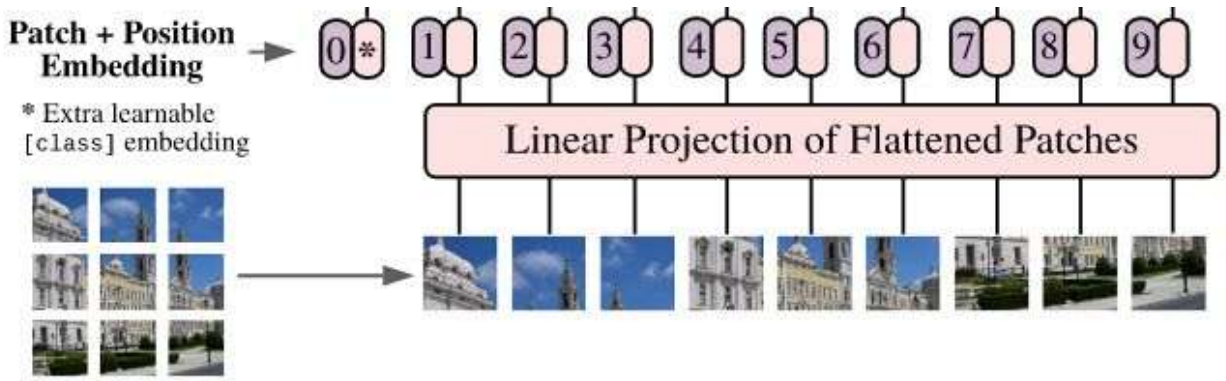
ViT: Putting it all together

1. Divide image into $P \times P$ patches
2. Create sequence of length $N = HW/P^2$
3. Flatten the patches and map to D dimensions with a trainable linear projection
4. Add a learned 1-D position embedding

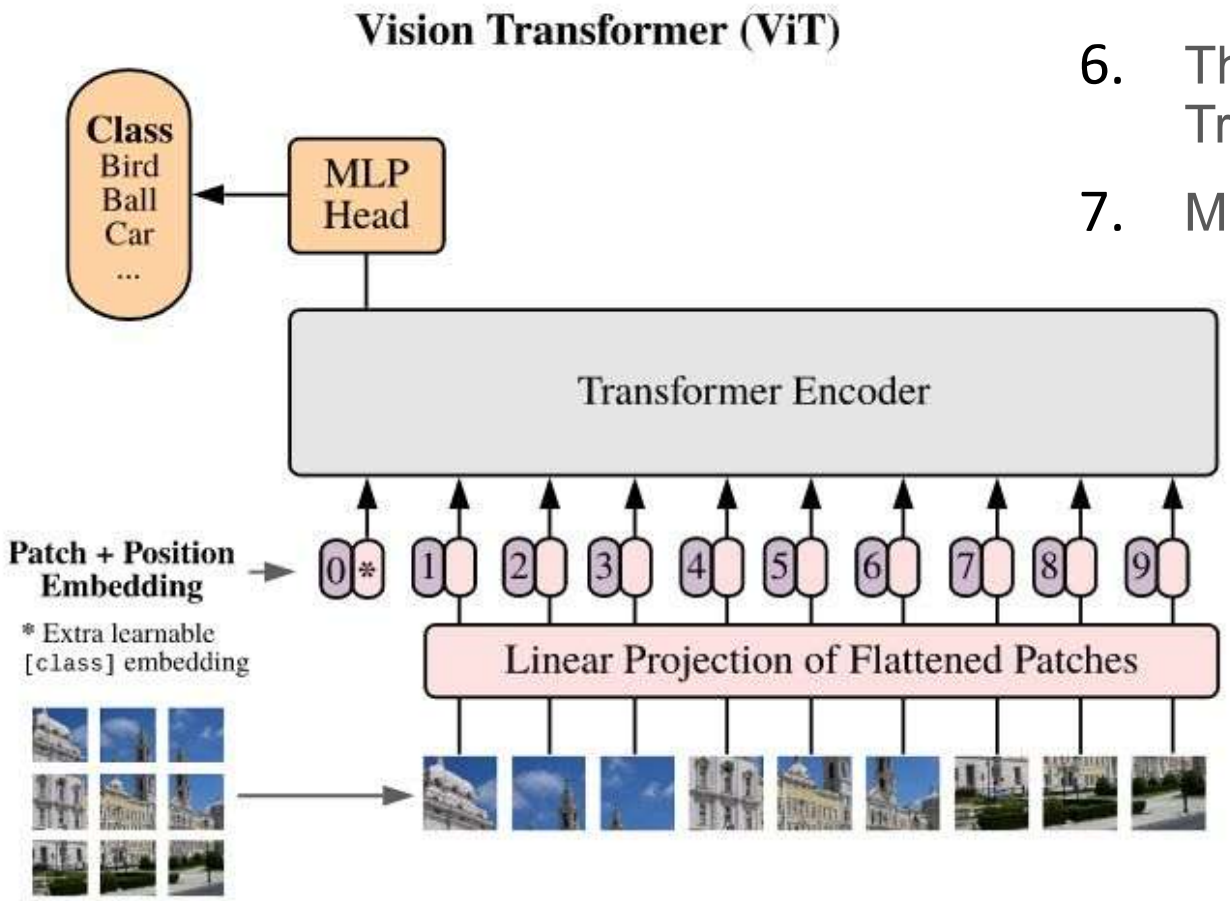


ViT: Putting it all together

1. Divide image into $P \times P$ patches
2. Create sequence of length $N = HW/P^2$
3. Flatten the patches and map to D dimensions with a trainable linear projection
4. Add a learned 1-D position embedding
5. Include a learnable [class] embedding



ViT: Putting it all together



6. Then through a multi-layered Transformer encoder to a
7. MLP classification head.

ViT Training Datasets & Model Variants

Dataset	# Classes	# Images
ILSVRC-2012	1K	1.3M
ImageNet-21K	21K	14M
JFT	18K	303M

Model	Layers	Hidden size D	MLP size	Heads	Params	
ViT-Base	12	768	3072	12	86M	Same as BERT
ViT-Large	24	1024	4096	16	307M	Same as BERT
ViT-Huge	32	1280	5120	16	632M	New for ViT

Notation: ViT-L/16 -- ”Large” variant with 16×16 input size.

Note: $16 \times 16 \times 3 = 768$

The DETR Model

Set Prediction Loss for Object Detection

- Infer a fixed-size set of N predictions, where N is much larger than the number of objects in an image
- Use Hungarian Algorithm to find a bipartite matching with the lowest matching cost:

$$\hat{\sigma} = \arg\min_{\sigma \in \mathfrak{S}_N} \sum_{i}^{N} \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$$

- Matching cost accounts for class probability and the predicted box:

$$-\mathbb{1}_{\{c_i \neq \varnothing\}} \hat{p}_{\sigma(i)}(c_i) + \mathbb{1}_{\{c_i \neq \varnothing\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\sigma(i)})$$

Set Prediction Loss for Object Detection

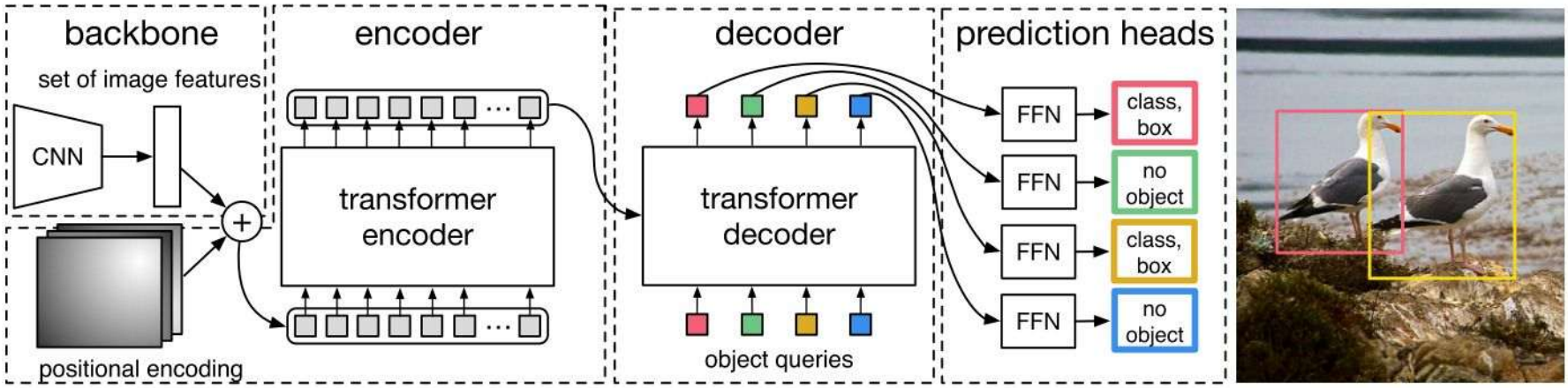
- Loss function: A Hungarian loss for all pairs matched in the previous step using NLL and bounding box loss:

$$\mathcal{L}_{\text{Hungarian}}(y, \hat{y}) = \sum_{i=1}^{N} \left[-\log \hat{p}_{\hat{\sigma}(i)}(c_i) + \mathbb{1}_{\{c_i \neq \varnothing\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\hat{\sigma}}(i)) \right]$$

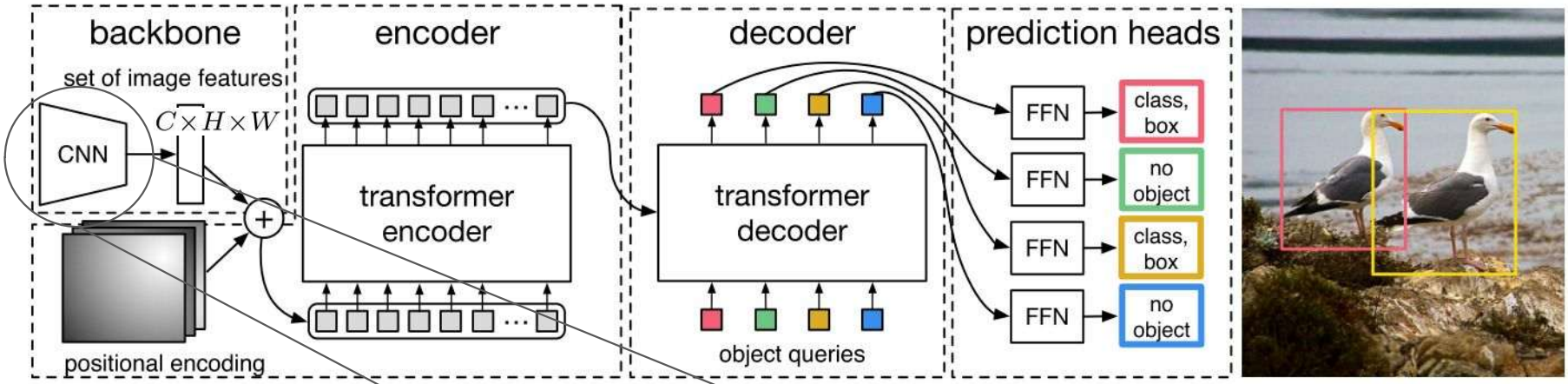
- Bounding box loss use a combination of the L1 loss and the generalized IoU loss that is scale-invariant:

$$\lambda_{\text{iou}} \mathcal{L}_{\text{iou}}(b_i, \hat{b}_{\sigma(i)}) + \lambda_{\text{L1}} ||b_i - \hat{b}_{\sigma(i)}||_1$$

DETR Architecture



DETR Architecture



- CNN Backbone:

$$f \in \mathbb{R}^{C \times H \times W}$$

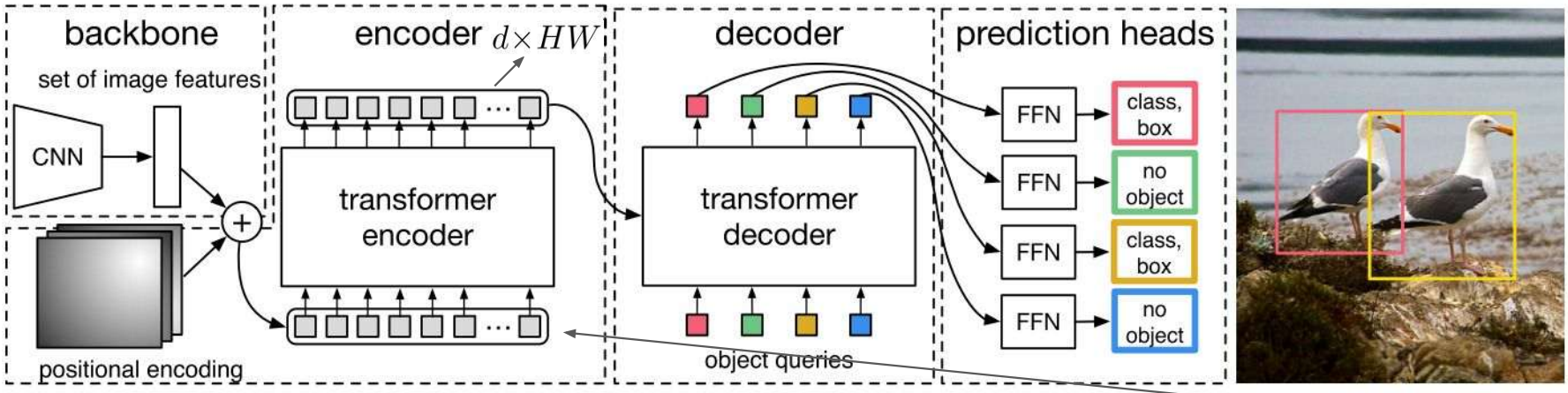
Activation Map $C \times H \times W$

$$x_{\text{img}} \in \mathbb{R}^{3 \times H_0 \times W_0}$$

Typically:

$$C = 2048 \quad H, W = \frac{H_0}{32}, \frac{W_0}{32}$$

DETR Architecture



- Transformer Encoder:
  - Reduce channel dimension (1x1 Convolution)
  - Flatten features into a sequential feature map
  - Add positional encodings to input of each attention layer
  - A multi-head self-attention module and a feed forward network

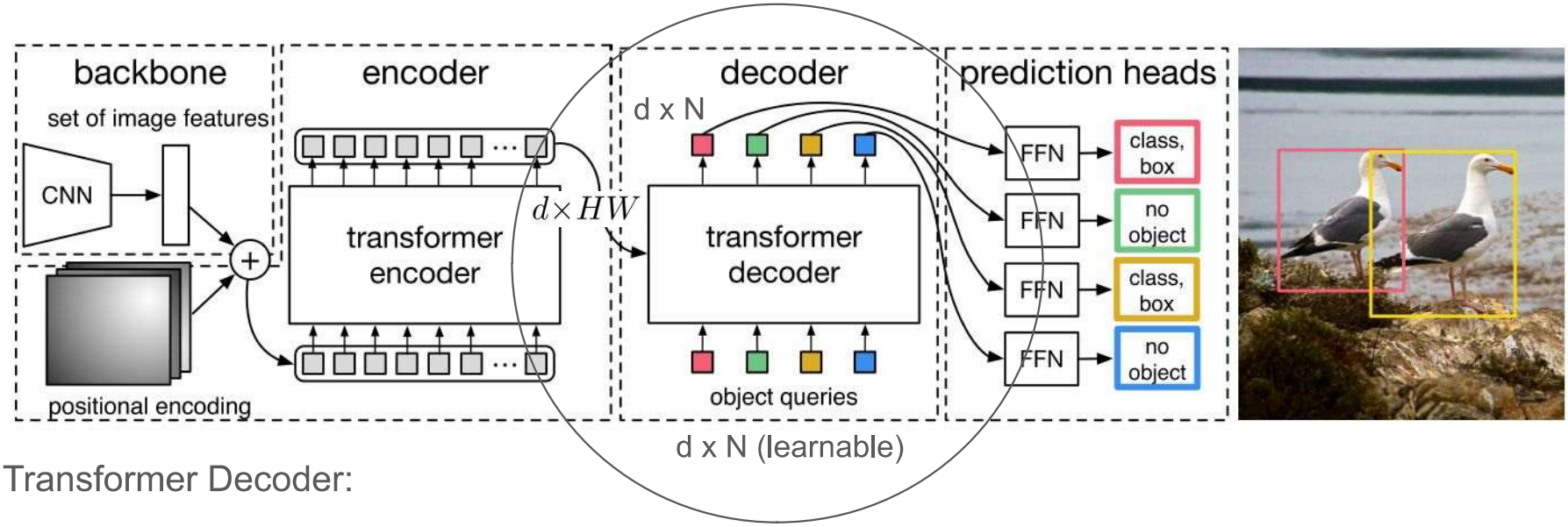
$$C \times H \times W \longrightarrow z_0 \in \mathbb{R}^{d \times H \times W}$$

$$d \times HW$$

$$+$$

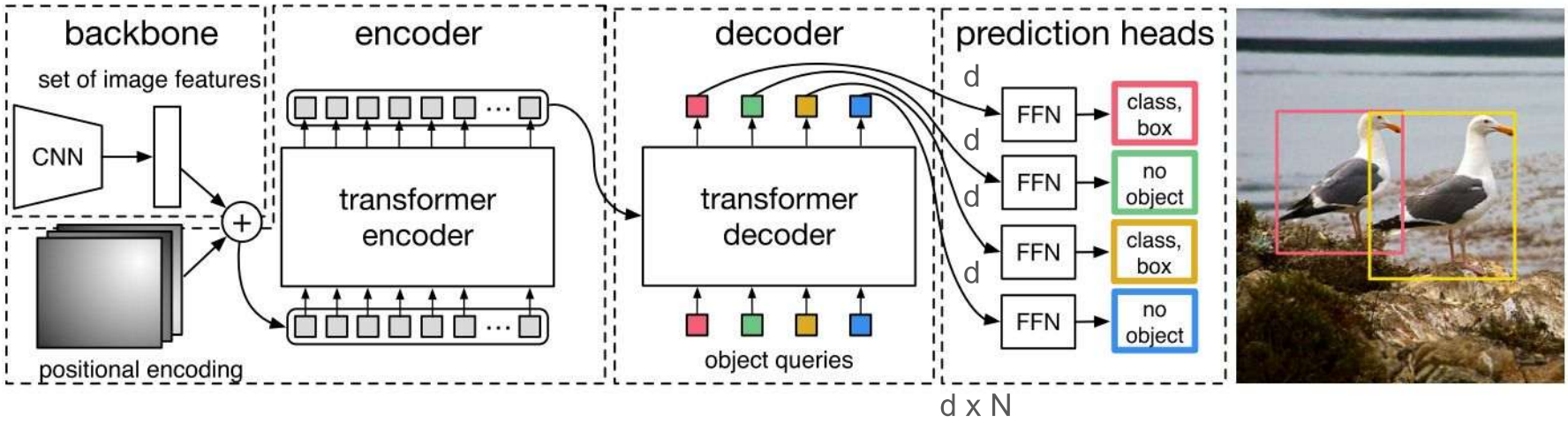
Positional Encoding

DETR Architecture



- Transformer Decoder:
  - Use encoded representations, d x HW embeddings, from encoder as key and value
  - Add N learnt positional encodings (object queries) to input of each attention layer
  - Transforms N object queries into N output embeddings in parallel (non-autoregressive)
  - Trained with auxiliary decoding loss to improve training

DETR Architecture



- Prediction Feed-forward networks (FFNs):
  - For normalized center coordinates: 3-layer perceptron with ReLU activation, hidden dim d
  - For class prediction: a linear projection layer with softmax
  - Class prediction also predicts “no object”

Detection Baselines

- Faster R-CNN
  - Features explored: Dilated C5 (DC5), Feature Pyramid Network (FPN), and ResNet-101 backbone with FPN (R101-FPN)
  - Stronger Faster R-CNN baselines:
    - Longer training (like for transformers)
    - Same random crop augmentation
    - Add generalized IoU to the box loss
- Can DETR perform comparably to ResNet under similar settings?

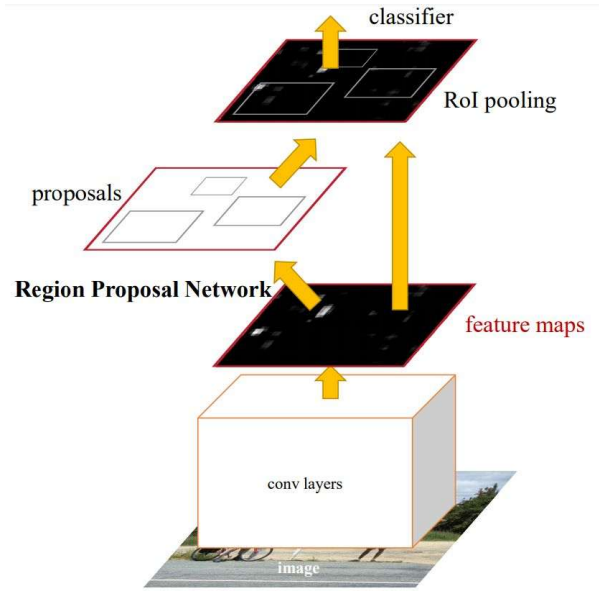


Figure 2 of Ren et al., 2016

Detection Metrics

- AP (Average Precision)
  - Precision: True Positives / (True Positives + False Positives)
    - Correct predictions out of all predictions
  - Recall: True Positives / (True Positives + False Negatives)
    - Correct predictions out of all objects in ground truth
  - Average Precision: area beneath the precision-recall curve

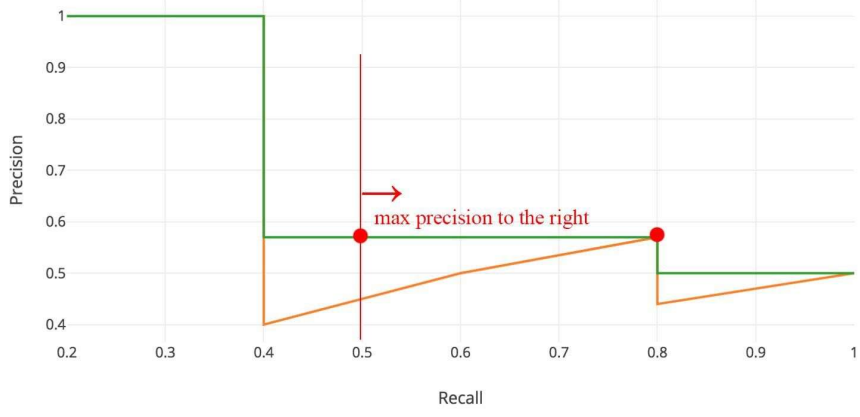


Image Source

Detection Metrics

- AP (Average Precision)
  - Intersection-over-Union (IoU): Area of Overlap / Area of Union
    - Measures how much bounding box prediction intersects with ground truth
  - AP: Average AP at IoU = 50%, 5%, and 95%.
  - AP_{50}: Only bounding box with IoU = 50% is counted as true positive
    - Similarly for AP_{75}
  - AP_S, AP_M, AP_L: AP based on objects of different sizes
    - Refer to: https://cocodataset.org/#detection-eval

Detection Results

Model	GFLOPS/FPS	#params	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Faster RCNN-DC5	320/16	166M	39.0	60.5	42.3	21.4	43.5	52.5
Faster RCNN-FPN	180/26	42M	40.2	61.0	43.8	24.2	43.5	52.0
Faster RCNN-R101-FPN	246/20	60M	42.0	62.5	45.9	25.2	45.6	54.6
Faster RCNN-DC5+	320/16	166M	41.1	61.4	44.3	22.9	45.9	55.0
Faster RCNN-FPN+	180/26	42M	42.0	62.1	45.5	26.6	45.4	53.4
Faster RCNN-R101-FPN+	246/20	60M	44.0	63.9	**47.8**	**27.2**	48.1	56.0
DETR	86/28	41M	42.0	62.4	44.2	20.5	45.8	61.1
DETR-DC5	187/12	41M	43.3	63.1	45.9	22.5	47.3	61.1
DETR-R101	152/20	60M	43.5	63.8	46.4	21.9	48.0	61.8
DETR-DC5-R101	253/10	60M	**44.9**	**64.7**	47.7	23.7	**49.5**	**62.3**